Same Parts, Different Arrangements Matched Edits for Budgeted Anomaly Inspection

Anonymous CVPR submission

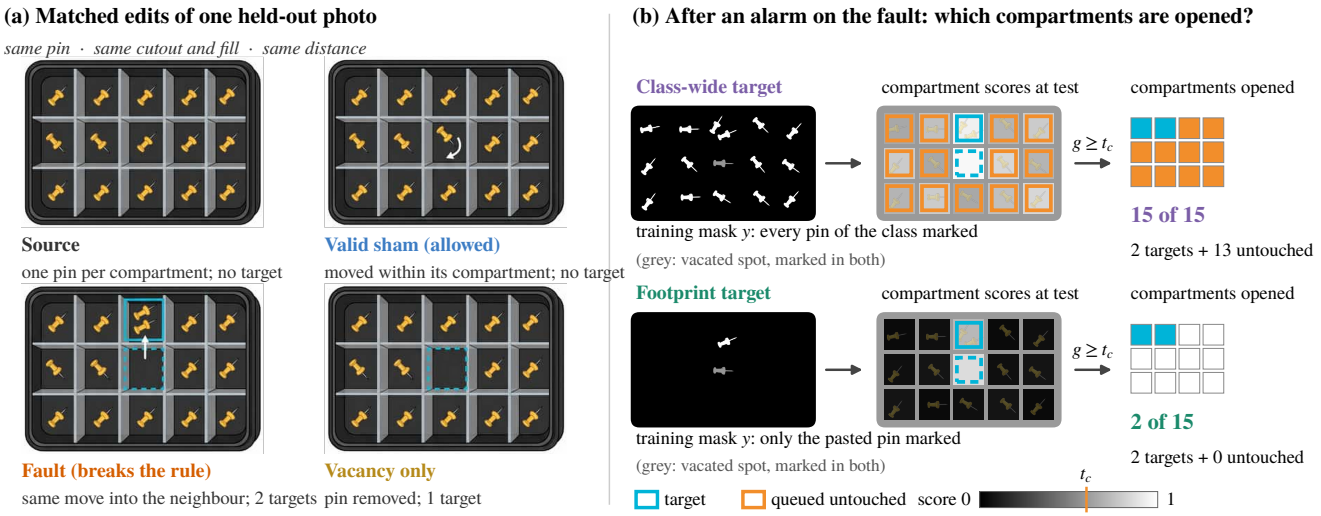


Figure 1. **Where should an inspector look after an alarm?** (a) Matched edits of one tray (rule: one pin per compartment): the same pin moves the same distance within its compartment (valid) or into an occupied neighbour (fault). Cyan: compartments to check (dashed: vacated; solid: recipient). (b) Schematic: marking every pin of the inserted class in training makes the whole tray score high, so all 15 compartments are opened; marking only the pasted pin opens just the 2 targets. Measured at 95% recall: 11.07 vs. 0.20–0.45 untouched compartments (Tab. 1). Tray art created in BioRender.

Abstract

An anomaly detector can correctly flag a faulty product and still send the inspector to check most of its parts. We measure this cost on a tray of 15 compartments that must each hold one pushpin. From 138 held-out photographs we build matched edits. The same pin moves the same distance, either within its compartment, which is allowed, or into an occupied neighbour, which breaks the rule. The edit fixes the two compartments an inspector must check. A recent logical anomaly detector flags every faulty tray. However, when its threshold is set to find 95% of those compartments, it also sends the inspector to 11.07 of the 13 untouched ones on average. We fine-tune only its final scoring network, changing what training marks as anomalous: the moved part alone instead of every part of the same type. This reduces the extra checks to 0.20 and 0.45 compartments in two training runs, at the same recall and with every fault still flagged. The same matched design shows which branch of the detector tells a fault from a valid edit under a second rule, and tests a graph model's sensitivity to edits it should ignore.

1. Introduction

Visual inspection systems are usually judged by whether they flag defective products. On a production line the alarm is only the first step: someone then has to find what is wrong. A tray of pushpins with one empty compartment and one doubled compartment contains the right number of pins, so only their *arrangement* is wrong. A useful detector should flag the tray and point to the two compartments that need attention. If it also highlights the other 13, the inspector gains little from the map.

Logical anomaly benchmarks such as MVTec LOCO [5] score detection by image-level AUROC and localization by pixel overlap. Neither number says how many places an in-

spector must check to find nearly every problem. Measuring that requires knowing exactly which locations are wrong. Natural logical defects rarely provide this: the extra screw in a bag can be any of the identical screws [6].

We obtain exact targets by intervention (Figure 1a). On held-out normal photographs we cut out an intact pin and move it, either within its compartment (a *valid sham*) or the same distance into an occupied neighbour (a *fault*). A third edit only removes the pin (*vacancy only*). Every arm shares the cutout, the background fill and the displacement. A difference between the arms can therefore only come from the rule violation, and the fault fixes the two compartments an inspector must open.

With these edits we turn a score map into an inspection decision with an explicit cost (Sec. 3). A scene alarm decides whether a tray is opened. A compartment cutoff, fixed on development photos, then decides which compartments are queued. We report the share of target compartments found (*recall*) and the number of untouched compartments queued (*cost*).

Our study centres on SALAD [6], a recent logical anomaly detector whose composition branch scores predicted part maps. Our contributions are:

- **A matched inspection test.** Valid shams, faults and vacancy-only edits with identical pixels moved, plus a recall–cost endpoint calibrated on development data (Sec. 3).
- **A diagnosis of the released model.** It detects all 138 faults and ranks them above their shams. At 95% target recall it also queues 11.07 of 13 untouched compartments per fault (Sec. 4.1).
- **A one-line change to the training target.** Marking only the pasted part reduces that cost to 0.20–0.45 compartments in two seeds, with paired intervals far from zero (Sec. 4.2).
- **Two transfer checks.** A second rule on juice bottles identifies which SALAD branch carries the evidence, and a graph check isolates a degree pathway in a masked link predictor (Secs. 5 and 6).

2. Related work

Logical anomaly detection. Industrial anomaly detection was first benchmarked on local structural defects such as scratches and dents [4], where memory banks of patch features [16] and student–teacher networks [3] perform strongly. MVTec LOCO adds *logical* defects that break explicit constraints on counts, positions or combinations of parts [5]. It scores localization with saturated per-region overlap (sPRO) to tolerate ambiguity. Component-based methods segment parts and compare attributes, correspondences or executable rules [9, 13, 15, 21]. SALAD learns from composition maps alongside local and global RGB branches [6]. We do not propose a new detector. We ask what one of these detectors costs an inspector once it has fired.

Synthetic anomalies as supervision. Many detectors train a discriminative head on synthetic defects. CutPaste pastes image patches [11]. DRAEM blends textures through Perlin masks and learns a segmentation of the pasted region [19]. LogicAL synthesizes logical defects for localization [22], and SALAD edits composition maps by inserting and relabelling parts [6]. How the target mask is drawn matters as much as which edits are made: it tells the head which pixels to call anomalous. We hold the edits fixed and compare two target policies.

Matched and counterfactual evaluation. Contrast sets [7] and counterfactually augmented data [10] test language models on minimally different inputs with different labels. Winoground pairs captions that share words but change relations [18]. Such pairs expose shortcut features that ordinary test sets reward [8]. Our valid shams play the same role for anomaly maps: they change the pixels as much as the fault but break no rule.

Calibrated alarms and graph readouts. We set alarms with split-conformal cutoffs from normal photographs [1, 2], which bound the normal alert rate, and fix compartment cutoffs on development edits. For graphs, target-link masking and degree effects are known pitfalls in link prediction [20, 23]. Our graph check measures how masking changes the signal available when two graphs have identical degree sequences.

3. Matched edits and the inspection decision

3.1. Setting and notation

A photo x_i shows a tray of $|\mathcal{E}| = 15$ compartments in a 5×3 grid. The published rule requires exactly one pin per compartment [5]. A part extractor maps the photo to a six-class part map $m_i \in \{0,\dots,5\}^{256\times256}$ (Figure 2). The detector's composition branch f returns a score map $S_i = f(m_i) \in [0,1]^{256\times256}$; higher means more anomalous. For a compartment $e \in \mathcal{E}$ we write $S_i|_e$ for the scores of its pixels. The compartment boundaries are registered once from the released maps.

3.2. Matched edits

We use all 138 held-out normal pushpin photographs. Using the released class mask and a two-pixel RGB halo, we cut out one intact pin and translate it without resampling. From each photo we make four variants (Figure 1a, Figure 3):

- **Source**: the untouched photo.
- **Valid sham**: the pin moves within its own compartment, so every compartment still holds one pin.

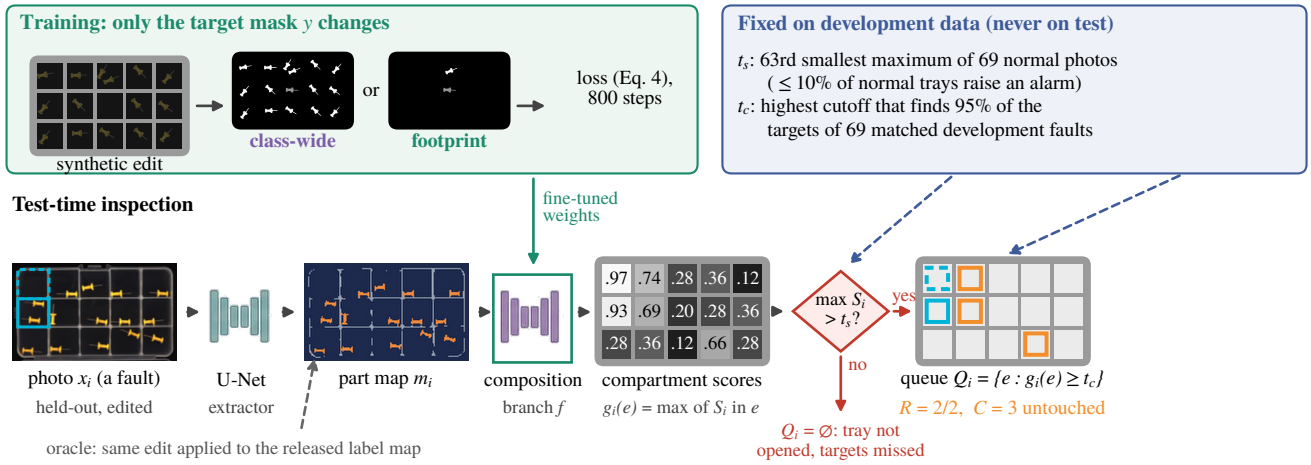


Figure 2. **Pipeline.** Training (green) changes only the target mask. At test, a part map comes from a U-Net (*extracted*) or from the edited label map (*oracle*); the scene alarm (t_s) and the compartment cutoff (t_c) are fixed on development data (blue). The values are illustrative and match the example under Equation (3) ($R = 2/2$, $C = 3$). Network icons created in BioRender.

- **Fault**: the pin moves by the same squared displacement into a face-adjacent, occupied compartment. Its source compartment a_i is now empty and the recipient b_i holds two pins.
- **Vacancy only**: the pin is removed and its support filled, with no paste.

The sham and the fault use the same cutout, the same fill (the median of the local dark background) and the same paste area. They change exactly the same number of RGB pixels. Collisions, divider clearance and background agreement are checked before any score is computed. The moves span 19–51 pixels.

Each variant has a *target set* $\mathcal{T}_i$, the compartments an inspector must check. For a fault, $\mathcal{T}_i = \{a_i, b_i\}$. For a vacancy, $\mathcal{T}_i = \{a_i\}$. A source or sham has $\mathcal{T}_i = \emptyset$, so every compartment queued there is a false mark. The vacancy-only edit keeps the missing half of the fault, so comparing it with the fault isolates the effect of the extra pin.

We score each variant through two interfaces with the same detector weights. The *oracle* interface moves the pin in the released label map. The *extracted* interface re-segments every RGB variant, including the source, with an independent U-Net trained only on normal training photos (97.3% development pin IoU). The extracted path is the realistic one, since a deployed system never sees the true labels. Unless stated otherwise we report it.

3.3. From a score map to an inspection queue

An inspector first decides whether to open a tray and then which compartments to check. We model both steps with cutoffs fixed before testing (Algorithm 1).

Step 1: scene alarm. The tray raises an alarm if its highest pixel score exceeds a scene cutoff t_s:

$$\text{alarm}_i = \mathbb{1}\big[\max_p S_i(p) > t_s\big]. \tag{1}$$

We set t_s to the 63rd smallest of the maxima on 69 untouched validation photos. This is a strict split-conformal cutoff [2] at which at most 10% of normal trays raise an alarm.

Step 2: compartment scores and queue. Each compartment gets the maximum score inside it, $g_i(e) = \max S_i|_e$. With a compartment cutoff t_c, the queue is

$$\mathcal{Q}_i = \begin{cases} \{e \in \mathcal{E} : \ g_i(e) \geq t_c\} & \text{if alarm}_i = 1, \\ \emptyset & \text{otherwise.} \end{cases} \tag{2}$$

A tray without an alarm is never opened, so its targets count as missed.

Step 3: recall and cost. Over n faults, target recall R is the share of target compartments in the queue, and cost C is the mean number of untouched compartments queued per fault:

$$R = \frac{1}{2n}\sum_{i=1}^{n} |\mathcal{Q}_i \cap \mathcal{T}_i|, \qquad C = \frac{1}{n}\sum_{i=1}^{n} |\mathcal{Q}_i \setminus \mathcal{T}_i|. \tag{3}$$

For example, a fault whose queue holds both targets and three other compartments contributes $2/2$ to recall and 3 to cost. The cost ranges from 0 to 13.

Step 4: choosing t_c without test data. We fix t_c on 69 matched *development* faults, built the same way from validation photos, and never on test. The *95% cutoff* is the

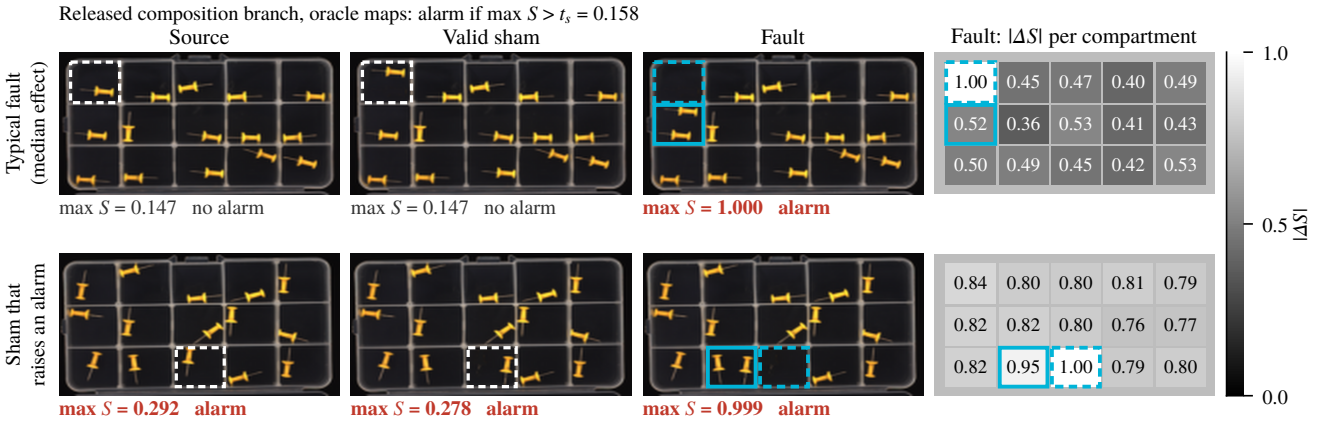


Figure 3. **Real matched edits** (oracle maps). Top: median released fault-minus-sham difference. Bottom: the first sham that raises an alarm; its source does too. White dashed: compartment of the moved pin; cyan: fault targets. Right: per-compartment $|S^{\text{fault}} - S^{\text{source}}|$; in the bottom row every compartment changes. Photos: MVTec LOCO AD, CC BY-NC-SA 4.0.

highest threshold that reaches 95% target recall there. With $2 \times 69 = 138$ development targets this is the 132nd largest target score, since $\lceil 0.95 \cdot 138 \rceil = 132$. A second, *fixed-cost cutoff* allows at most 0.5 untouched compartments per development fault. It is the 35th largest untouched score, since $\lfloor 0.5 \cdot 69 \rfloor + 1 = 35$. Both cutoffs are fitted separately for each map and interface. On test we report the achieved R and C, and the alarm rates on sources, shams, faults and vacancies.

Ranks without cutoffs. Some systems show the inspector a fixed number of compartments instead. We therefore also report *top 2*, the share of faults whose two highest-scoring compartments are both targets, and the mean rank of the second target. Ties are broken by compartment index.

A region-overlap baseline. For comparison with sPRO [5, 6] we sweep a common threshold over all compartments. Positives are the target compartments, and negatives are the untouched fault compartments plus every compartment of the source and sham. We integrate target recall over the false-compartment rate from 0 to 0.05 and divide by 0.05 (*region area*). Unlike (R, C), this sweep uses test data and ignores the scene alarm. It summarizes ranking quality, not the cost at a fixed operating point.

3.4. What the detector is trained to mark

SALAD trains its composition branch on synthetic edits of normal part maps. A discriminative U-Net outputs logits z_p and is fitted to a binary target mask y:

$$\mathcal{L} = 5 \cdot \frac{1}{|P|} \sum_p \mathrm{FL}(z_p, y_p) + \frac{1}{|P|} \sum_p \left| \sigma(z_p) - y_p \right|, \quad (4)$$

Algorithm 1 Calibrated inspection queue

Require: map model f; cells $\mathcal{E}$; normal validation photos $\mathcal{V}$; development faults $\mathcal{D}$ with targets $\mathcal{T}_j$

$t_s \leftarrow$ 63rd smallest of $\{\max_p f(m_v)(p)\}_{v \in \mathcal{V}}$ ▷ $|\mathcal{V}| = 69$

$G \leftarrow \{g_j(e) : j \in \mathcal{D},\ e \in \mathcal{T}_j\}$, with $g_j(e) = 0$ if j has no alarm

$t_c \leftarrow$ the $\lceil 0.95\,|G| \rceil$-th largest value of G ▷ 95% cutoff

for each test photo i **do**

  $S_i \leftarrow f(m_i)$; **if** $\max_p S_i(p) \le t_s$ **then** $\mathcal{Q}_i \leftarrow \emptyset$

  else $\mathcal{Q}_i \leftarrow \{e \in \mathcal{E} : \max S_i|_e \ge t_c\}$

return R and C from Equation (3)

where FL is the sigmoid focal loss with $\alpha = 0.25, \gamma = 2$ [12], σ is the logistic function and P is the set of pixels. The target y decides which pixels are called anomalous. When SALAD inserts a part of class k, producing the edited map m', the released code marks *every* pixel of that class [6]:

$$y^{\text{cw}}(p) = \mathbb{1}[m'(p) = k] \qquad \text{(class-wide)}. \quad (5)$$

The rationale is that an extra screw in a bag could be any screw. In a tray, however, only one compartment is overfull. We therefore also consider marking only the donor's footprint F, the pixels actually pasted:

$$y^{\text{fp}}(p) = \mathbb{1}[p \in F] \qquad \text{(footprint)}. \quad (6)$$

For relocations both targets also mark the vacated footprint, and shams always get $y \equiv 0$. Same-image relabelling targets are unchanged. The two policies thus differ only in the insertion part of the mask. In seed 1 a relocation marks 3,823 pixels on average under the class-wide policy and 461 under the footprint policy.

Paired fine-tuning. Each fine-tune starts from the released composition weights and freezes the reconstruction autoencoder. Each batch of four holds two clean maps, one SALAD-style synthetic edit, and one *rule-aware* edit (relocation, sham or vacancy) built from official training photos. We run 800 Adam steps and keep the last checkpoint, for two seeds. Within a seed, the two policies see identical inputs, edits, steps, loss weights and pixel weights; only y differs. The rule is used to build training edits, but not at test time, where the model sees only a part map.

3.5. Pre-specified comparison

The primary contrast is the difference in cost C between the footprint and class-wide policies on extracted maps. Before testing we fixed what counts as useful:

- at least one fewer untouched compartment, with a 95% interval excluding zero;
- achieved recall $R \geq 90\%$, at most 5 points below the control;
- fault and vacancy alarm rates $\geq 95\%$, at most 3 points below;
- at most 5 points more sham alarms than the control.

A mechanistic reading also requires a competent class-wide control on development data: alarm rates of at least 90% and a compartment AUROC of at least 90%. All intervals use 4,000 bootstrap draws of original photos, resampling all variants, policies and interfaces together. They condition on the fitted models and cutoffs. Two seeds and two fixed halves of the photos (63 and 75) check stability.

4. Results on the pushpin tray

4.1. What the released detector does well

The released composition branch raises an alarm on all 138 faults and all 138 vacancies, on both interfaces. It scores every fault above its matched sham (138/138; exact 95% interval 97.4–100%). On oracle maps the affected-compartment AUROC is 48.6% on the source, i.e. chance, and rises to 98.2% after the fault.

Its ranking is good too. On extracted maps both targets are among the top two compartments in 77.5% of faults and among the top three in 94.2%; the second target appears at rank 2.43 on average. Scoring compartments by their 99th percentile instead of their maximum raises the top-two rate to 87.0% (+9.4 points [4.3, 14.5]) with the same alarm and weights.

Where the budget changes the picture. At the 95% development cutoff the released maximum finds 95.7% of target compartments but queues 11.07 of the 13 untouched compartments per fault (Tab. 1). In other words, the inspector checks almost the whole tray. The 99th percentile, better at rank two, is no better here (11.41 at 96.0%). At the 98% cutoff the released map queues 12.56. The fixed-cost cutoff tells the same story from the other side: holding the cost to 0.30 compartments, the released map finds only 81.9% of targets (Figure 5d). A readout that wins at a fixed list length can therefore lose at a fixed recall.

Table 1. **Pushpin inspection on extracted maps** (138 test photos). Every row raises an alarm on all 138 faults and 138 vacancies. *Sham*: valid shams with an alarm. *Top 2*: both targets in the two highest compartments. *2nd*: mean rank of the second target. R and C use the 95% development cutoff (Equation (3)). *Area*: region area at false-compartment rate ≤ 0.05, in percent. Oracle maps are in the supplement.

Map, compartment score	Sham	Top 2	2nd	R (%)	C	Area
Released, max	7	77.5	2.43	95.7	11.07	81.6
Released, 99th pct.	7	87.0	2.32	96.0	11.41	77.3
Class-wide, seed 1	19	0.0	15.00	97.1	13.00	39.3
Footprint, seed 1	9	87.7	2.13	95.3	**0.20**	**96.3**
Class-wide, seed 2	22	0.0	15.00	96.4	13.00	39.4
Footprint, seed 2	20	74.6	2.28	96.0	**0.45**	89.6

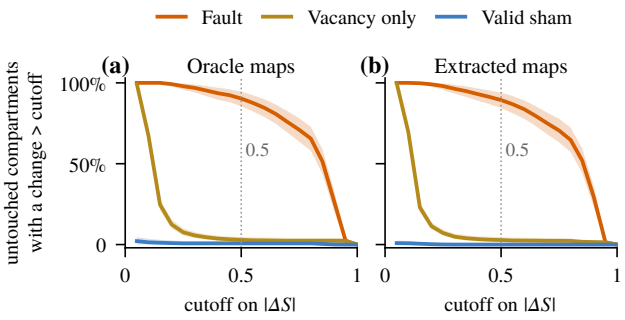


Figure 4. **The released map responds across the tray.** Share of the 13 untouched compartments with a pixel whose score changes by more than the cutoff, with 95% intervals over photos.

Why the queue is long. The fault changes the released map almost everywhere (Figure 4). On oracle maps, 90.3% of untouched compartments contain a pixel whose score changes by at least 0.5. The changes are sharp peaks rather than broad areas: only 5.4% of their pixels change that much. The vacancy-only edit, which leaves the same hole, moves only 2.9% of untouched compartments, and the sham 0.7%. The fault-minus-vacancy difference is 87.4 points [82.4, 91.9] on oracle maps and 86.6 [81.4, 91.2] after extraction. The extra pin, not the hole, makes the map light up across the tray. This is what the class-wide target teaches: when a pin is inserted, every pin is anomalous.

4.2. Footprint targets shorten the queue

Main result. Fine-tuning with footprint targets reduces the 95%-recall queue from 11.07 untouched compartments

to 0.20 and 0.45 in the two seeds, at 95.3% and 96.0% recall (Tab. 1). The paired reductions against the release are 10.88 [10.12, 11.56] and 10.62 [9.86, 11.31] compartments. Against the class-wide fine-tunes, the pre-specified control, they are 12.80 [12.72, 12.87] and 12.55 [12.45, 12.65] (Figure 6). Every fault and vacancy still raises an alarm. The recall differences have intervals that include zero and stay above the 5-point floor. The reduction holds on oracle maps (10.30 and 10.04 against the release) and on both fixed halves of the photos (-12.87 and -12.75 for seed 1, -12.63 and -12.48 for seed 2). Figure 5 shows a typical case: the released map queues all 13 untouched compartments; the footprint map queues none.

Other operating points. At the fixed-cost cutoff the footprint heads reach 100.0% and 98.2% recall at achieved costs of 0.38 and 0.57, against 81.9% at 0.30 for the release. In a descriptive sweep at an equal realized cost of 0.50 compartments, recall is 82.6% for the release and 100.0% and 97.1% for the footprint heads, gains of $+17.4$ [13.4, 21.0] and $+14.5$ [9.8, 18.8] points. The region area rises from 81.6% to 96.3% and 89.6%, so the ranking-based summary agrees in direction. It does not, however, reveal how short the calibrated queue becomes.

What does not improve. Two results are seed-dependent. First, sham alarms: seed 1 raises 9 against 7 for the release ($+1.4$ points [-3.6, 7.2]), but seed 2 raises 20 ($+9.4$ [2.2, 16.7]). Against the class-wide control, the pre-specified comparison, neither seed exceeds the 5-point guard (-7.2 and -1.4). Second, the top-two rate rises to 87.7% in seed 1 but falls to 74.6% in seed 2, below the release.

The class-wide control. The class-wide fine-tunes keep every alarm but light up the whole tray: all 13 untouched compartments are queued and the top-two rate is 0. Their development compartment AUROC is about 48%, so they fail the competence check of Sec. 3.5. The comparison with the class-wide arm shows that, within this recipe, the target alone produces the difference. The comparison with the release measures the gain of the whole adaptation, which also adds rule-aware edits, and cannot be credited to the mask alone.

4.3. A continuation closer to the released recipe

To separate the target from the rule-aware edits, we also ran a continuation that uses only SALAD's original synthetic edit families, replacing the rule-aware slot with a clean map. Its class-wide seed 2 passes the competence check (97.3% development AUROC) and reproduces the released behaviour: 88.8% of untouched compartments change by at least 0.5. Footprint targets reduce that share to 1.3% and the queue from 11.50 to 7.07, a paired reduction of 4.43 [3.72, 5.14] at 96.0% and 95.3% recall. Seed 1 shows a similar reduction (4.28) but its control misses the AUROC floor. The footprint target thus shortens the queue even without rule-aware training, and the rule-aware edits give the lowest costs.

5. Branch choice under two rules

SALAD fuses three branches: a local RGB appearance map, the composition map studied above, and a global RGB score that pools student features over the scene and over regions given by the part map [6]. We replayed the full released system. On 91 native logical pushpin faults and 138 normal photos it reproduces the published 100.0% logical AUROC.

Pushpins. The matched shams reveal a cost that this AUROC hides (Figure 7b). Fusion scores every fault above its sham, yet raises an alarm on 130/138 extracted shams and 75/138 sources; 55 shams alarm whose source did not. The local RGB branch alone alarms on 128 shams and only 9 sources. Both arms carry the same compositing, so the local RGB branch responds to the edit itself, not to the rule. The composition branch alarms on 7 shams. An explicit one-pin-per-compartment rule on the same extracted maps alarms on no sham and catches every fault. This is the rule the application supplies.

Juice bottles. MVTec LOCO also requires that the icon on a juice bottle's upper label match the liquid [5]. We keep 89 of 94 eligible normal test bottles and paste intact icons from a disjoint bank of training photos into the same label region (Figure 7a). A same-type icon is a valid sham; another type is a fault. Both use the same offset, blending and footprint. Their mean blended RGB changes differ by at most half a grey level. The liquid, bottle and part map are unchanged, so the composition branch returns identical maps for all three variants and ties exactly. The global RGB branch scores all 89 faults above their shams and alarms on every fault but only 14 shams. Its fault-minus-sham alarm gap is $+84.3$ points [76.4, 91.0] (Figure 7d), slightly above fusion's $+79.8$ [70.8, 87.6]. The local RGB branch puts the upper label first among five bottle regions on 74.2% of faults, but also on 85.4% of shams. As a scene detector it alarms on 25 faults and 28 shams.

The two rules thus favour different branches. Composition evidence decides placement in a tray. Global appearance decides a label–liquid match that the part map cannot express. Matched shams make this choice measurable. The pushpin target adaptation itself is evaluated only on pushpins.

6. A graph check on response evidence

The same matched logic applies to graphs of parts. We build gland graphs from GlaS histology [17] and cardiac structure graphs from HVSMR-2.0 [14]. We then apply

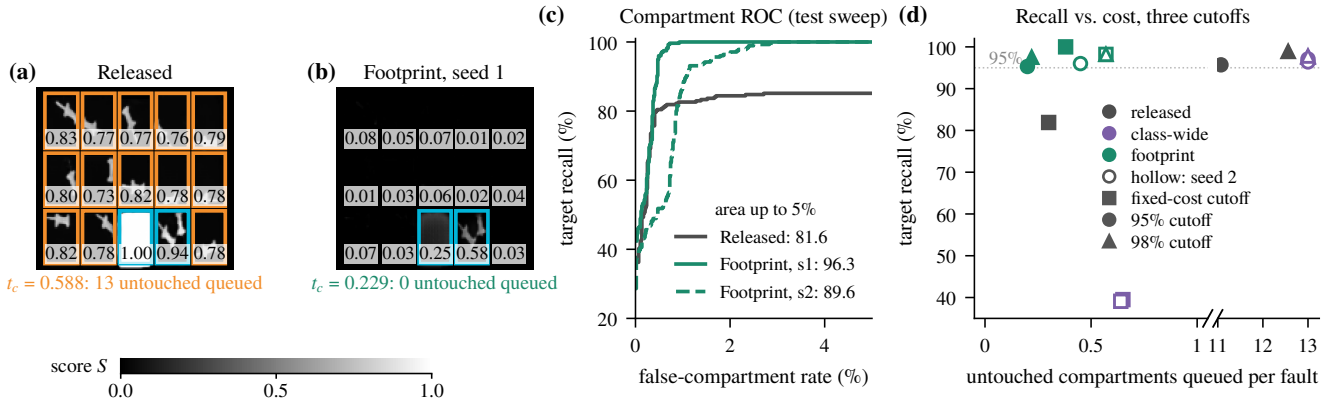


Figure 5. **Footprint supervision empties the queue.** (a,b) Extracted maps of the median fault (greyscale) with each compartment's maximum; cyan: targets, orange: untouched compartments at or above the locked 95% cutoff t_c. (c) Compartment ROC from a test sweep, with the area up to 5% from Tab. 1 (class-wide curves, area 39.3–39.4, omitted). (d) Recall against cost at the fixed-cost (square), 95% (circle) and 98% (triangle) development cutoffs; hollow: seed 2; the axis is broken between 1 and 11.

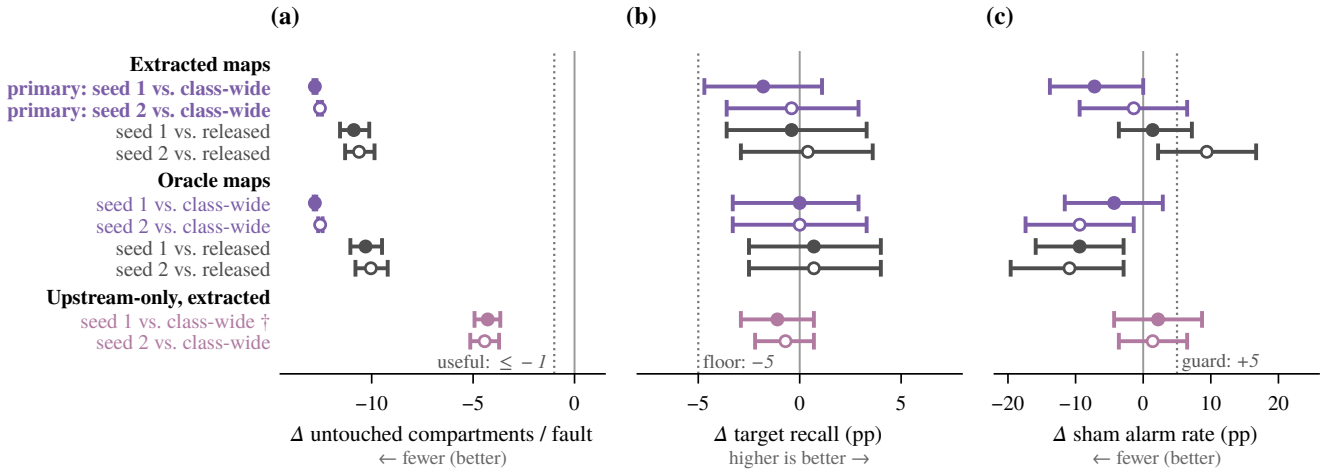


Figure 6. **Paired differences at the locked cutoffs**, footprint minus each comparator, with 95% intervals; hollow: seed 2. Bold: the pre-specified primary contrast. Dotted: pre-specified thresholds. †: this seed's control fails the competence check (Sec. 4.3). Values: supplement Tabs. 3 and 8.

degree-preserving switches: two links are removed and two added so that every node keeps its degree. Readouts are scored on how well they rank the four switched pairs.

Geometry helps ranking. In GlaS, adding local gland spacing to typed centroid distance improves the edited-graph AUROC by 15.3 points [10.9, 20.1]. The gain persists on extra length-matched switches and under joint matching of raw and local distances (+16.8 [11.3, 23.0]). These tiers reuse donors and contain no pathological labels.

Why a masked GNN responds. A link predictor with logit z_e and observed link state $a_e \in \{0, 1\}$ has the Bernoulli loss $\mathrm{softplus}(z_e) - a_e z_e$. Flipping a_e changes this loss even if the evidence z_e is fixed. A pair MLP sees only geometry and roles, so its probability cannot change, and it ranks by response at chance (Figure 8b). A query-masked neighbourhood model has another route. It removes the queried link before computing the endpoint's degree, so the degree it sees is $d_i - a_e$. A switch flips a_e on the four target pairs, so their masked degrees change even though the full graphs have identical degree sequences. Target-link exclusion is known to create such degree effects [23].

The masked GNN recovers 87.5% of switched pairs in its top four by absolute change in probability (Figure 8c). Holding its degree inputs fixed reduces this to 33.7%. We also refit the network with mean neighbourhood aggregation and no degree inputs on the same four GlaS folds. The refit reaches 95.7–98.2% development AUROC on normal links, but its top-four recovery falls to 46.6%, 40.9 points lower

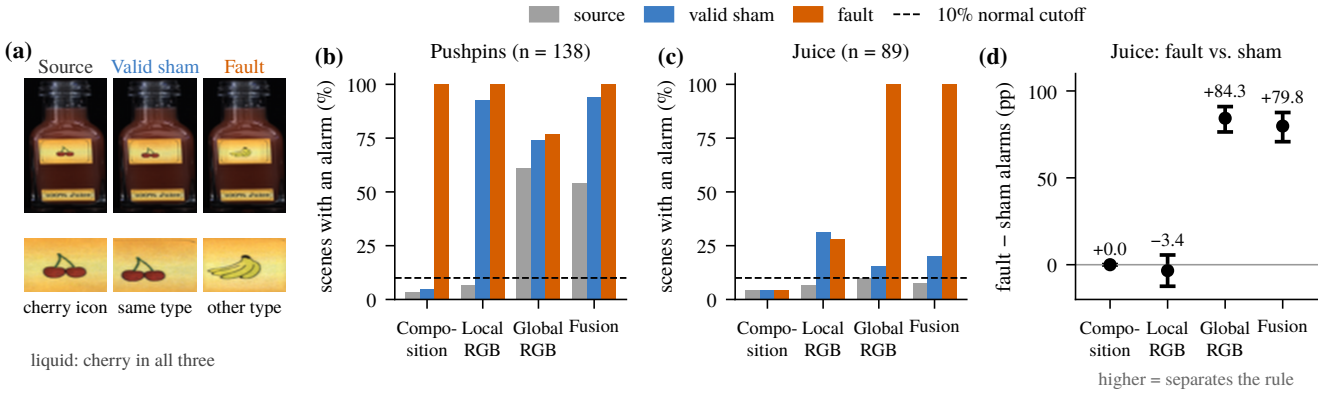


Figure 7. **Which SALAD branch separates a fault from a valid edit?** (a) Juice edit: a same-type (valid) or other-type (fault) icon, pasted identically. (b,c) Scenes alarmed per branch at its own 10% normal cutoff (dashed); on pushpin composites, global RGB and fusion also alarm on most untouched sources. (d) Juice fault-minus-sham alarm gap, 95% intervals. Photos: MVTec LOCO AD, CC BY-NC-SA 4.0.

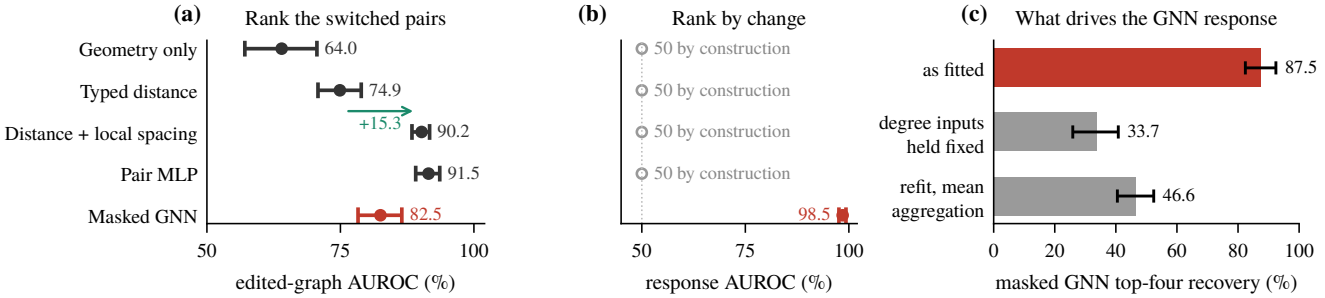


Figure 8. **Graph check on 78 GlaS fields (12 donors).** (a) Edited-graph AUROC; local spacing adds 15.3 points to typed distance. (b) AUROC of the change between matched graphs; hollow: 50 by construction. (c) Top-four recovery of the masked GNN as fitted, with degree inputs fixed, and refit with mean aggregation. Donor-paired 95% intervals.

[32.8, 49.8]. Mean aggregation still allows indirect cues such as an empty neighbourhood. The clamp and the refit point to a degree pathway, so recovering known graph edits should not be read as validity.

7. Limitations

The strongest result concerns two compartments in a registered tray. Both seeds improve the calibrated high-recall queue. The best-two ranking and sham specificity, however, depend on the seed. Training used edits built from a known rule, so the gain describes that setting. Our interventions are synthetic composites on held-out photos. They match pixels changed, cutout and displacement, but they are not physical rearrangements, and physical tray identities are unavailable. Intervals condition on the fitted heads and cutoffs. The class-wide rule-aware controls fail the competence check, so mechanism claims rest on the upstream continuation, where one seed passes. The juice test covers one further rule and the graph check one masking design. Native LOCO results and a single exploratory cardiac query are in the supplement.

8. Conclusion

Matched edits turn a logical anomaly benchmark into a measurement of inspection effort. A detector that catches every fault and ranks its targets well can still send the inspector to 11 of 13 untouched compartments at high recall; image AUROC, top-k rates and region areas do not show this, while the recall–cost pair at a development cutoff does. The training target is the lever: marking every part of the inserted class teaches that one extra pin makes all pins anomalous, which suits interchangeable parts in a bag but not parts tied to locations in a tray. Marking only the pasted part reduces the cost to under half a compartment. Valid shams also reveal which branch carries the evidence under a new rule, and expose a degree pathway in a masked graph model. We hope that reporting recall together with inspection cost, against matched valid edits, becomes standard practice for logical anomaly detection.

LLM Usage Statement

OpenAI Codex was used to develop and implement the revision experiments, run and check the code, inspect public sources and data provenance, produce figures and tables from saved outputs, and draft and edit the main text and supplementary material. Anthropic Claude was used to restructure and edit the text and to redraw the figures from the recorded outputs. Figures use deterministic plotting and the documented photograph composites. The authors take full responsibility for the content.

References

[1] Anastasios N. Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023. 2

[2] Stephen Bates, Emmanuel Candès, Lihua Lei, Yaniv Romano, and Matteo Sesia. Testing for outliers with conformal p-values. *The Annals of Statistics*, 51(1):149–178, 2023. 2, 3

[3] Kilian Batzner, Lars Heckler, and Rebecca König. EfficientAD: Accurate visual anomaly detection at millisecond-level latencies. In *IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 128–138, 2024. 2

[4] Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger. MVTec AD – a comprehensive real-world dataset for unsupervised anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9592–9600, 2019. 2

[5] Paul Bergmann, Kilian Batzner, Michael Fauser, David Sattlegger, and Carsten Steger. Beyond dents and scratches: Logical constraints in unsupervised anomaly detection and localization. *International Journal of Computer Vision*, 130 (4):947–969, 2022. 1, 2, 4, 6

[6] Matic Fučka, Vitjan Zavrtanik, and Danijel Skočaj. SALAD – semantics-aware logical anomaly detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 21843–21852, 2025. 2, 4, 6

[7] Matt Gardner, Yoav Artzi, Victoria Basmov, Jonathan Berant, Ben Bogin, Sihao Chen, Pradeep Dasigi, Dheeru Dua, Yanai Elazar, Ananth Gottumukkala, Nitish Gupta, Hannaneh Hajishirzi, Gabriel Ilharco, Daniel Khashabi, Kevin Lin, Jiangming Liu, Nelson F. Liu, Phoebe Mulcaire, Qiang Ning, Sameer Singh, Noah A. Smith, Sanjay Subramanian, Reut Tsarfaty, Eric Wallace, Ally Zhang, and Ben Zhou. Evaluating models' local decision boundaries via contrast sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1307–1323, 2020. 2

[8] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. 2

[9] Chenrui Ji, Jiafan Lu, Zilu Luo, Feng Wan, Qiuhui Chen, and Weimin Zhong. SGDB: Semantic-geometric decoupling and evidence binding for logical anomaly detection. *arXiv preprint arXiv:2610.34897*, 2026. 2

[10] Divyansh Kaushik, Eduard Hovy, and Zachary C. Lipton. Learning the difference that makes a difference with counterfactually-augmented data. In *International Conference on Learning Representations*, 2020. 2

[11] Chun-Liang Li, Kihyuk Sohn, Jinsung Yoon, and Tomas Pfister. CutPaste: Self-supervised learning for anomaly detection and localization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9664–9674, 2021. 2

[12] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988, 2017. 4

[13] Tongkun Liu, Bing Li, Xiao Du, Bingke Jiang, Xiao Jin, Liuyi Jin, and Zhuo Zhao. Component-aware anomaly detection framework for adjustable and logical industrial visual inspection. *Advanced Engineering Informatics*, 58:102161, 2023. 2

[14] Danielle F. Pace, Hannah T. M. Contreras, Jennifer Romanowicz, Shruti Ghelani, Imon Rahaman, Yue Zhang, Patricia Gao, Mohammad Imrul Jubair, Tom Yeh, Polina Golland, Tal Geva, Sunil Ghelani, Andrew J. Powell, and Mehdi Hedjazi Moghari. HVSMR-2.0: A 3D cardiovascular MR dataset for whole-heart segmentation in congenital heart disease. *Scientific Data*, 11:721, 2024. 6

[15] Yun Peng, Xiao Lin, Nachuan Ma, Jiayuan Du, Chuangwei Liu, Chengju Liu, and Qijun Chen. SAM-LAD: Segment anything model meets zero-shot logic anomaly detection. *Knowledge-Based Systems*, 314:113176, 2025. 2

[16] Karsten Roth, Latha Pemula, Joaquin Zepeda, Bernhard Schölkopf, Thomas Brox, and Peter Gehler. Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14318–14328, 2022. 2

[17] Korsuk Sirinukunwattana, Josien P. W. Pluim, Hao Chen, Xiaojuan Qi, Pheng-Ann Heng, Yun Bo Guo, Li Yang Wang, Bogdan J. Matuszewski, Elia Bruni, Urko Sanchez, Anton Böhm, Olaf Ronneberger, Bassem Ben Cheikh, Daniel Racoceanu, Philipp Kainz, Michael Pfeiffer, Martin Urschler, David R. J. Snead, and Nasir M. Rajpoot. Gland segmentation in colon histology images: The GlaS challenge contest. *Medical Image Analysis*, 35:489–502, 2017. 6

[18] Tristan Thrush, Ryan Jiang, Max Bartolo, Amanpreet Singh, Adina Williams, Douwe Kiela, and Candace Ross. Winoground: Probing vision and language models for visio-linguistic compositionality. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5238–5248, 2022. 2

[19] Vitjan Zavrtanik, Matej Kristan, and Danijel Skočaj. DRAEM – a discriminatively trained reconstruction embedding for surface anomaly detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8330–8339, 2021. 2

[20] Muhan Zhang and Yixin Chen. Link prediction based on graph neural networks. In *Advances in Neural Information Processing Systems*, 2018. 2

[21] Ximiao Zhang, Min Xu, and Xiuzhuang Zhou. LogiCo: A

unified framework for logical and structural anomaly detection. *arXiv preprint arXiv:2606.28688*, 2026. 2

[22] Ying Zhao. LogicAL: Towards logical anomaly synthesis for unsupervised anomaly localization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 4022–4031, 2024. 2

[23] Jing Zhu, Yuhang Zhou, Vassilis N. Ioannidis, Shengyi Qian, Wei Ai, Xiang Song, and Danai Koutra. Pitfalls in link prediction with graph neural networks: Understanding the impact of target-link inclusion and better practices. In *ACM International Conference on Web Search and Data Mining*, pages 994–1002, 2024. 2, 7